

FarmUnity — Technology Stack, Algorithms & Flowchart

Comprehensive Architectural Rationale, Mathematical Formulations & Component Call Tree

1. Technology Stack Selection & Engineering Rationale

Every tier was chosen specifically to solve high-reliability, zero-downtime, and rural connectivity constraints:

Layer	Technology Selected	Specific Version	Engineering Rationale & Justification
Backend Framework	Spring Boot (Java 21 LTS)	v3.2.5	Enterprise transactional integrity, strict compile-time type safety, built-in validation, and virtual threads support for high concurrent throughput.
Persistence & ORM	Spring Data JPA / Hibernate	v6.4.4	Object-Relational mapping with declarative transactions (@Transactional) and strict @Version optimistic locking to avoid contract race conditions.
Database Engine	Embedded H2 in PostgreSQL Mode	v2.2.224	Zero-setup local execution requiring no external containers or daemon services, fully compatible with ANSI SQL and production PostgreSQL.
Database Migrations	Flyway Core	v9.22.3	Deterministic, version-controlled schema evolution ensuring idempotent table creation and zero drift between environments.
Security & Tokens	Spring Security + JJWT	v0.11.5	Stateless HMAC-SHA256 JWT claims validation with BCrypt password hashing. Completely eliminates server session storage overhead.
Frontend Framework	React 18 + Vite 5	React 18 / Vite 5.4	Lightning-fast sub-second HMR dev server, compact ES module bundles, and concurrent DOM rendering for high-responsiveness.
Client State & Caching	TanStack React Query	v5.35	Automated background cache invalidation, stale-while-revalidate data fetching, and declarative query state management.
Offline Persistence	Dexie.js (IndexedDB wrapper)	v4.0.1	Guarantees rural offline draft listing creation. Local transactions are automatically queued and synced when connection recovers.
Geospatial UI	React-Leaflet + OpenStreetMap	v4.2.1 / Leaflet 1.9	Open-source map rendering with zero proprietary API billing keys, displaying multi-stop waypoint collection trajectories.
Styling & Icons	Tailwind CSS + Lucide React	Tailwind 3.4 / Lucide	Utility-first design system with custom keyframe orbit animations inspired by Farmsent.io and featherweight SVG iconography.

2. Core Algorithms & Mathematical Formulations

A. Greedy Haversine Spatial Micro-Clustering

Used in: CooperativeFormationService.java

Purpose: Solves the geographical aggregation of fragmented smallholder farmers into single bulk transport batches.

Formula: Computes great-circle angular distance between coordinate pairs:

$$a = \sin^2(\Delta\phi/2) + \cos(\phi_1) * \cos(\phi_2) * \sin^2(\Delta\lambda/2)$$

$$c = 2 * \text{atan2}(\sqrt{a}, \sqrt{1-a}), \text{Distance} = R * c \text{ (where } R = 6371 \text{ km)}$$

Selection Policy: Filters listings matching crop, variety, and verified moisture bounds. Sorts listings by distance from buyer destination and minimum price floor. Iteratively accumulates farmer lots until requirement volume is met within max_distance_km.

B. Bi-Lateral Threshold Clearing Price Algorithm

Used in: ThresholdPricingService.java

Purpose: Eliminates speculative middleman broker cuts by finding fair clearing prices between farmer reserve floors and institutional price ceilings.

Formula: $\text{OverlapLow} = \max(\text{FarmerFloor}_i)$, $\text{OverlapHigh} = \text{BuyerCeiling}$

Validation: If $\text{OverlapLow} \leq \text{OverlapHigh}$, a viable commercial zone exists.

Clearing Price: $P_{\text{agreed}} = (\text{OverlapLow} + \text{OverlapHigh}) / 2$

Result: Surpluses are split equitably between producer cooperative and buyer.

C. Mathematical Net Payout & Settlement Formulation

Used in: SettlementService.java

Purpose: Enforces deterministic payment calculations strictly using verified tare/gross weighbridge quantities.

Formulas:

$\text{Gross_Amount} = \text{Verified_Qty} * P_{\text{agreed}}$

$\text{Transport_Deduction} = \text{Verified_Qty} * \text{Transport_Rate_Per_Kg}$

$\text{Platform_Fee} = \text{Gross_Amount} * 0.02$ (2% platform operational fee)

$\text{Net_Farmer_Payout} = \text{Gross_Amount} - \text{Transport_Deduction} - \text{Platform_Fee}$

Integrity Rule: Declared quantity is never referenced during settlement.

D. Exponential Moving Average Reliability Rating

Used in: TrustTierTransitionService.java

Purpose: Quantitative performance scoring adjusting participant trust levels based on historical fulfillment accuracy.

Formula: $\text{Reliability}_t = (\text{Reliability}_{\{t-1\}} * 0.70) + (\text{OrderScore} * 0.30)$

Tiers: Score ≥ 95 : TRUSTED; ≥ 85 : ESTABLISHED; ≥ 70 : VERIFIED; < 70 : NEW.

E. Cryptographic SHA-256 State Integrity Chaining

Used in: HashUtil.java, AuditLogListener.java

Purpose: Tamper-evident ledger hashing linking all state transitions.

Formula: $\text{Hash} = \text{SHA-256}(\text{entityType} + ':' + \text{entityId} + ':' + \text{eventType} + ':' + \text{actorId} + ':' + \text{timestamp} + ':' + \text{details})$

3. Platform Call Tree & Component Flowchart

Below is the hierarchical call sequence across controllers, services, repositories, and UI views:

[1. PRODUCER FLOW]

FarmerDashboard.jsx / CreateListingPage.jsx

--> api/listings.js: create() [or Dexie useDraftQueue.js if offline]

--> ProduceListingController: POST /api/listings

--> ProduceListingService.createListing()

--> ProduceListingRepository.save(ProduceListing[status=PENDING_INSPECTION])

--> AuditEventPublisher: ListingCreated

[2. PHYSICAL VERIFICATION FLOW]

VerificationPage.jsx (Coordinator Field Device)

--> api/verification.js: recordVerification()

--> VerificationController: POST /api/verification/verify

--> VerificationService.verifyListing()

--> CropConfigRepository.findByCropName() [Moisture min/max check]

--> VerificationRecordRepository.save()

--> ProduceListingRepository: update(verifiedQty, status=INSPECTED)

--> AuditEventPublisher: ListingVerified

[3. CLUSTERING & COOPERATIVE BATCH FLOW]

BatchFormationPage.jsx / BuyerDashboard.jsx

--> api/batches.js: formBatch()

--> CooperativeBatchController: POST /api/batches/form

--> CooperativeFormationService.formBatch()

--> HaversineUtil.calculateDistance() [Micro-clustering within max_distance_km]

--> ThresholdPricingService.calculateOverlap() [OverlapLow vs OverlapHigh]

--> CooperativeBatchRepository.save(status=FORMING)

```
--> BatchMembershipRepository.saveAll(farmers, status=PENDING)
--> AgreementRepository.save(Agreement[@Version=1, status=PENDING_APPROVAL])
```

[4. DEMOCRATIC CONSENT & ESCROW LOCK FLOW]

```
FarmerNegotiationPage.jsx --> api/agreements.js: submitFarmerDecision(ACCEPTED)
--> AgreementController: POST /api/agreements/{id}/farmer-decision
--> AgreementService.recordFarmerDecision() [All members accept -> status=AWAITING_FUNDING]
Buyer: CommitmentLedgerPage.jsx --> api/commitments.js: fund()
--> CommitmentLedgerController: POST /api/commitments/{id}/fund
--> CommitmentLedgerService.fundCommitment()
--> PaymentService.lockDeposit() [STATUS: FUNDED]
--> AgreementRepository: update(status=LOCKED_AND_FUNDED)
```

[5. LOGISTICS, DELIVERY & NET SETTLEMENT FLOW]

```
PickupPage.jsx --> LogisticsService.dispatch() [Waypoint pickup route]
MillingPage.jsx --> MillingService.process() [Yield shortfall tolerance check]
DeliveryConfirmationPage.jsx --> api/delivery.js: confirmDelivery()
--> DeliveryController: POST /api/delivery/confirm
--> DeliveryService.confirmDelivery()
--> SettlementService.executeSettlement()
--> Calculates Gross, Transport Deduction, 2% Fee, Net Payout
--> SettlementRepository.saveAll()
--> CommitmentLedgerRepository: update(status=RELEASED)
--> TrustTierTransitionService.recordSuccessfulDelivery()
--> AuditLogListener: SHA-256 Hash generated and saved to audit_logs
```